Reference-Set Size and Score Aggregation in TF–IDF Cross-Language Code Retrieval

Roman Tsoy • *Independent Research Project* • *29 October 2026*

An Empirical Study on AtCoder and CoREB

Abstract

When searching for code written in another programming language, a retrieval system may have several implementations of the same task to use as references. More references provide more examples of how a task can be implemented, but the benefit depends on how their similarity scores are combined. This study examines reference-set size, score aggregation, and reference filtering using a TF–IDF baseline on an AtCoder-derived Java–Python corpus and the cross-language code-to-code subset of CoREB.

On a fixed AtCoder evaluation cohort, query-weighted Success@1 for mean aggregation is 0.5070 with all available references, compared with 0.2046 with one reference. On AtCoder, mean aggregation also outperforms maximum aggregation across the tested multi-reference settings. On CoREB, moving from one to two references improves the five metrics tested inferentially for both aggregation strategies: 95% source-problem cluster-bootstrap intervals for the k=2 minus k=1 contrasts exclude zero for Hit@1, Hit@5, Recall@10, MRR, and nDCG@10 under both Max and Mean. In a separate filtering experiment, the apparent benefit of restricting the corpus to implementations marked as having passed tests becomes small when both conditions rank the same candidate groups.

The results show that reference-set size can substantially affect this lexical retrieval baseline in the tested AtCoder cohort, and that moving from one to two references improves retrieval metrics in the tested CoREB group-level protocol. Aggregation and filtering effects depend on the dataset and evaluation design. Because relevance is defined differently across the two datasets, their absolute scores are not directly comparable. CoREB provides evidence only for the one-versus-two-reference contrast; it does not support conclusions about larger reference sets.

Keywords: cross-language code retrieval; code search; TF–IDF; reference implementations; score aggregation; information retrieval

1. Introduction

Finding a program that solves the same task in another language can help developers reuse code, compare implementations, or work with a codebase written in an unfamiliar language. For example, a developer may have a Python solution and want to locate a Java implementation of the same algorithm. In a large repository, manually inspecting every candidate is impractical, so the system must rank candidates by relevance.

This is harder than ordinary text search: two programs can solve the same task while sharing few tokens because they use different variable names, control-flow structures, or library functions. The research question grew out of learning Java and Python side by side. Tasks intended to produce the same result often looked quite different because the languages encourage different syntax and implementation choices. This raised a practical question: can a retrieval system find related implementations even when their surface forms differ?

Prior work has explored several ways to represent and compare code. COSAL combines token-level, structural, and behavioral evidence for cross-language search [1]. Another relevant line of work asks how to combine similarity scores from multiple reference implementations. WOJR compares maximum and average similarity when recommending related programming problems [2]. These studies show that aggregation is not a new problem by itself. The question examined here is narrower: how does the number of reference implementations affect retrieval, and does the relative behavior of maximum and mean aggregation change as more references are added?

Suppose that ten Java solutions are available for a programming task. One solution may resemble a Python query in token usage, while another may use a different implementation style. If the retrieval system sees only one reference, its result can depend on which example was selected. Additional references may capture more variation, but they also create a decision about how to combine the resulting similarity scores.

This study compares two simple strategies. Maximum aggregation assigns a candidate the highest similarity score among its selected references. Mean aggregation averages similarity across those references. The maximum can preserve a strong match to one implementation, while the mean rewards candidates that remain similar across the set. Neither strategy is expected to be best in every setting; their behavior needs to be measured.

I chose TF–IDF as a deliberately simple baseline so that the experiment would focus on reference selection and score aggregation rather than on training or tuning a learned code model. This choice makes the comparisons easier to interpret, but it also limits what the system can recognize: the baseline measures lexical overlap, not program behavior. I evaluate the two strategies on an AtCoder-derived corpus of Python queries and Java candidate solutions, and on the cross-language code-to-code subset of CoREB [4]. The study addresses three research questions:

- RQ1. How does retrieval performance change as the number of reference implementations increases?
- RQ2. How do maximum and mean aggregation compare across reference-set sizes and datasets?
- RQ3. When filtering implementations by test status improves retrieval, does the improvement remain when the candidate group set is held fixed?

This study compares reference-set size and Max/Mean score aggregation on two datasets with different relevance definitions. It also examines whether the apparent benefit of filtering remains when the candidate groups are kept fixed. The goal is not to introduce a new retrieval architecture, but to understand how these design choices affect a simple lexical baseline.

2. Related Work

2.1. Cross-language code search

Mathew and Stolee introduced COSAL, which combines token similarity, structural similarity, and behavioral evidence to retrieve related code across languages [1]. Their evaluation includes Java and Python programs from AtCoder. COSAL is relevant both because it treats code as the search query and because it illustrates the limits of relying on a single notion of similarity. The present study uses a simpler lexical baseline and examines a different design choice: how to use multiple implementations of a candidate task.

2.2. Aggregating multiple implementations

Yoshimura et al. introduced WOJR, a system for recommending programming problems based on similarities between model answers and candidate reference code [2]. Their approach compares maximum and average similarity. The authors report that the better operator depends on the repository and experimental setting, and discuss the sensitivity of average aggregation to noisy references. This makes it important not to treat one operator as a universal default. Our experiments extend the comparison across several reference-set sizes and two datasets, while keeping the lexical scoring method fixed.

Table 1. How prior work relates to the present study.

Work	Main focus	Relationship to this study
COSAL [1]	Cross-language code search using token, structural, and behavioral evidence; includes Java/Python code.	Establishes the search problem and AtCoder context. This study narrows the method to a TF–IDF baseline to isolate reference-set effects.
WOJR [2]	Compares maximum and average similarity for recommending related programming problems.	Shows that Max-versus-average aggregation is not new by itself. This study varies reference-set size and adds a matched-candidate filtering control.
XLCoST [3]	Benchmark for cross-lingual code intelligence, including code-to-code tasks.	Provides related dataset context, but it is not part of the experiments reported here.
CoREB [4]	Code retrieval/reranking benchmark with document-level graded judgments and hard negatives.	Supplies an external test setting. This paper uses a custom problem-language grouping protocol, not official document-level metrics.

Note. This comparison identifies the specific relationship to the present study; it is not a claim that the listed projects exhaust the literature.

3. Experimental Method

3.1. Retrieval baseline

Programs are represented using TF–IDF features over code tokens and token bigrams. The tokenizer recognizes identifiers, numeric literals, common multi-character operators, and other non-whitespace symbols. Lowercasing is disabled to preserve identifier capitalization, and sublinear term-frequency scaling is enabled. TF–IDF serves as a deliberately simple lexical baseline: it avoids training a learned model and keeps the representation fixed while the experiment varies reference-set size and score aggregation. This design makes those choices easier to isolate, but it does not imply that TF–IDF is the strongest approach to code retrieval or that the same patterns would hold with semantic or learned representations.

For a query program q and a candidate implementation d, similarity is measured using cosine similarity between their TF–IDF vectors:

$$s(q, d) = (v_q \cdot v_d) / (||v_q|| \, ||v_d||)$$

Let R_g be the selected reference implementations for candidate group g. The two aggregation scores are:

$$S_max(q, g) = max \{ s(q, d) : d \in R_g \}$$

$$S_mean(q, g) = (1 / |R_g|) \Sigma \, s(q, d), \text{ for } d \in R_g$$

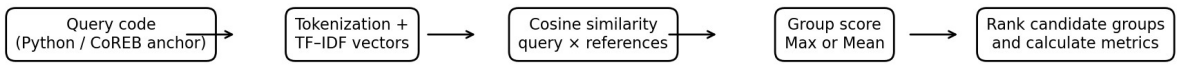


Figure 1. Retrieval pipeline used to compare reference-set size and score aggregation. The exact query and relevance protocols differ between AtCoder and CoREB.

Candidate groups are ranked by descending aggregate score. When a group has one selected reference, max and mean are identical.

3.2. AtCoder corpus and evaluation

The AtCoder experiment uses an AtCoder-derived archive of Java and Python solutions. The input archive is AtCoder.mongo.zip from the COSAL dataset record on Zenodo (record 4002869; DOI: 10.5281/zenodo.4002869) [5]. The archive used for this analysis matched the record's expected file size of 350,063,499 bytes and MD5 checksum 11e95fae502b9fffcd9771aa1d9cb01a; ZIP integrity testing found no damaged members. Programs are associated with task identifiers extracted from their paths. Python programs are used as queries and Java programs form the candidate reference sets. A candidate is considered relevant when it belongs to the same task as the query. This is a task-level proxy for relevance: it does not establish that the retrieved program is the closest semantic equivalent at the code-fragment level.

The mapped Java pool contains 18,454 unique candidate programs across 360 task groups. The fixed evaluation cohort contains 1,438 Python queries associated with 290 tasks and 18,116 Java candidate programs from 294 task groups. Candidate groups are restricted to those with at least ten Java implementations, allowing a comparison of multiple reference-set sizes while keeping the query and candidate cohorts fixed. For each candidate task group, the selected references are drawn from that group's eligible Java implementations. The ranked candidate groups remain the same across the k = 1, 5, 10, and all-reference conditions; changing k therefore changes the evidence used to score each group, rather than the set of groups being ranked. For the fixed-k conditions, the reference-selection seeds are 0, 1, 2, 3, and 4. Within each task group, a seed-specific shuffled order defines nested reference sets for k = 1, 5, and 10; the all-reference condition uses every available implementation in the group.

The main AtCoder table reports query-weighted Success@1, Success@5, Success@10, and mean reciprocal rank (MRR). Success@k is 1 when at least one relevant task group appears in the top k results and 0 otherwise; values are averaged over queries, and each table entry is then averaged across the five reference-selection seeds. MRR is the average reciprocal rank of the first relevant result. Separate paired bootstrap comparisons were calculated at the task level, giving each task equal weight in the inferential comparison. Consequently, the bootstrap effect estimates need not exactly equal differences between the query-weighted means in the descriptive table.

3.3. CoREB corpus and group-level evaluation

The CoREB experiment uses the cross-language code-to-code queries from release_v2603. The audited subset contains 190 queries, 1,744 corpus programs, and 999 query-document judgments: 429 positive judgments at relevance level 2 and 570 hard-negative judgments at level 1. Candidate documents are grouped by source problem and programming language. For each query, groups are restricted to the target languages listed for that query, and the query's anchor document is excluded from retrieval.

To keep relevance labels stable across reference-set sizes, the relevance grade for each problem-language group is fixed using the maximum judged relevance of any document in the complete group. Groups with a grade of at least 2 are treated as relevant. This defines a group-level retrieval task: a group may remain relevant even if the particular implementation selected as its reference is not itself the positively judged document. The procedure avoids changing a group's label when fewer references are selected, but it does not measure the relevance of each selected reference independently.

The reference-size experiment compares one randomly selected implementation with two references and all available references. The one-reference condition is averaged over ten random seeds (0 through 9). Almost all groups contain only two implementations, so the two-reference and all-reference conditions are effectively equivalent. Metrics include Hit@1, Hit@5, Hit@10, Recall@1, Recall@5, Recall@10, MRR, and binary group-level nDCG@10. Hit@k measures whether any relevant group occurs in the top k; Recall@k measures the share of relevant groups retrieved in the top k. The reported group-level nDCG is based on binary group relevance and must not be interpreted as official graded document-level CoREB nDCG.

3.4. Reference-filtering experiment

The filtering experiment uses CoREB's meta_passed field to distinguish implementations marked as having passed tests from those that did not. The unrestricted comparison contrasts a search over all groups with a search over groups containing at least one passed implementation. Because filtering removes groups, this comparison changes both the reference set and the candidate space.

A matched-candidate control isolates the reference-selection effect more directly. It retains the same groups in both conditions, then compares scores calculated from all implementations in each retained group with scores calculated only from passed implementations. Group relevance labels remain fixed. This control retains an average of 151.45 candidate groups per query and does not remove any relevant group under the study's group-level relevance definition. Because the ranked groups are held constant, differences in this control are more directly attributable to the reference composition than differences in the unrestricted comparison, which changes both the references and the candidate space.

3.5. Statistical analysis

For AtCoder, paired task-level bootstrap intervals were computed for changes in reference-set size and for mean-minus-max comparisons. Point estimates in the main performance table are query-weighted, while these inferential comparisons are task-weighted; both estimands are reported explicitly. The reference-size contrasts use 10,000 task-level bootstrap replicates with seed 20261028; the task-weighted mean-minus-max contrasts use 5,000 task-level replicates with seed 20261028. For CoREB, paired cluster-bootstrap intervals resample source problems and retain all queries associated with each sampled problem. For the reference-count contrasts, k=2 minus k=1 was evaluated separately for Max and Mean using 5,000 percentile-bootstrap replicates with random seed 20261031, with the 2.5th and 97.5th percentiles as interval bounds. The resampling units are the 61 meta_source_problem_id clusters. For the filtering analysis, the final cluster-bootstrap audit uses 5,000 percentile-bootstrap replicates and the same 2.5th/97.5th percentile bounds. Its base seed is 20261030; the five comparisons use seeds 20261030 through 20261034 in the order listed in the reproducibility README. For each comparison, random-reference seeds are averaged within query before differences are calculated. Comparisons are paired by query and resampled by source-problem cluster. The complete settings and all reported intervals are included with the release artifacts. These intervals quantify variability within the evaluated CoREB subset; they do not establish generalization to other datasets or protocols.

4. Results

4.1. Reference-set size and aggregation on AtCoder

Table 2 reports query-weighted results on the fixed AtCoder cohort, averaged over five reference-selection seeds. Increasing the number of reference implementations improves performance for both aggregation methods. Mean aggregation has the strongest results at every tested reference-set size greater than one.

Table 2. Query-weighted AtCoder performance

References	Aggregation	Success@1	Success@5	Success@10	MRR
1	Max / Mean	0.2046	0.3534	0.4334	0.2823
5	Max	0.3417	0.5469	0.6403	0.4435
5	Mean	0.3954	0.6074	0.6928	0.4955
10	Max	0.3932	0.6225	0.7143	0.5017
10	Mean	0.4572	0.6866	0.7597	0.5638
All	Max	0.4527	0.6759	0.7441	0.5559
All	Mean	0.5070	0.7392	0.8032	0.6128

With mean aggregation, query-weighted Success@1 rises from 0.2046 with one reference to 0.3954 with five, 0.4572 with ten, and 0.5070 with all available references. Query-weighted MRR increases from 0.2823 to 0.4955, 0.5638, and 0.6128. Success@5 rises from 0.3534 to 0.6074, 0.6866, and 0.7392; Success@10 rises from 0.4334 to 0.6928, 0.7597, and 0.8032. Figure 2 summarizes the Success@1 trend. These are descriptive averages over queries; the task-level bootstrap estimates reported below use equal weight per task and therefore target a different average.

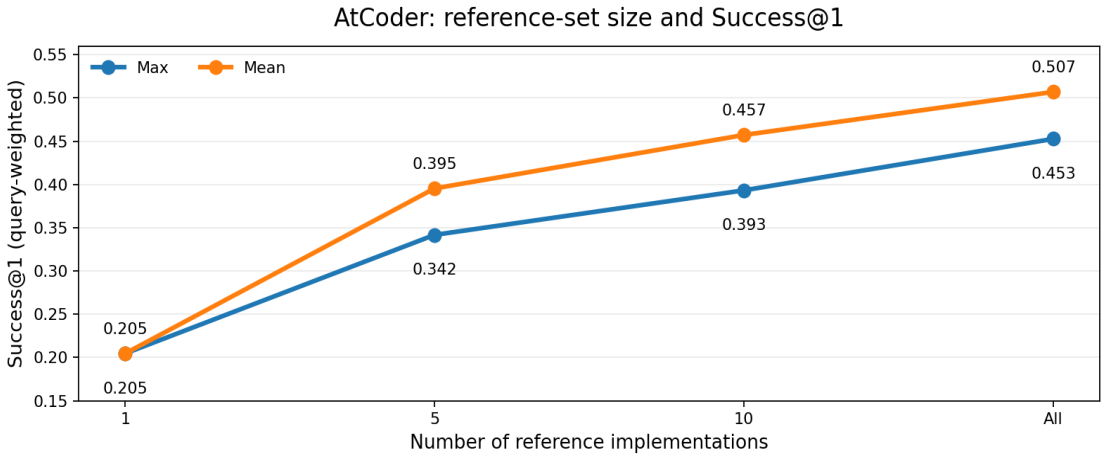


Figure 2. Query-weighted Success@1 by reference-set size on AtCoder. Values are taken from Table 2; the figure is descriptive and does not show confidence intervals.

The task-level bootstrap comparisons support the same overall pattern. For mean aggregation, increasing the reference set from one to five produced a task-weighted Success@1 gain of 0.1913 (95% CI [0.1701, 0.2128]) and an MRR gain of 0.2135 (95% CI [0.1951, 0.2317]). The gain from five to ten references was smaller but remained positive: +0.0629 for Success@1 (95% CI [0.0510, 0.0749]) and +0.0690 for MRR (95% CI [0.0590, 0.0784]). Increasing from ten references to all available references yielded a further Success@1 gain of 0.0493 (95% CI [0.0335, 0.0657]). The confidence intervals are task-weighted and should not be read as intervals for the query-weighted values in Table 2.

Mean aggregation also outperforms max aggregation in the task-level comparisons. With all references, the task-weighted difference in Success@1 (mean minus max) is 0.0584 (95% CI [0.0193, 0.0990]); the MRR difference is 0.0604 (95% CI [0.0292, 0.0919]). Within this evaluation cohort, the mean operator makes better use of multiple reference scores than simply retaining the maximum score.

4.2. CoREB reference-set size and aggregation

Table 3 shows the group-level CoREB point estimates, and Table 4 reports cluster-bootstrap differences between two references and one reference. The one-reference condition is averaged over ten random seeds. With two references, mean aggregation has higher Hit@1 and MRR, whereas max aggregation has higher Hit@5 and Recall@10. The point-estimate comparison between Max and Mean is mixed; the cluster-bootstrap intervals below test the effect of reference count within each aggregation strategy, not the difference between aggregation strategies.

Table 3. Group-level reference-size results

References	Aggregation	Hit@1	Hit@5	Hit@10	Recall@10	MRR	Group nDCG@10
1	Max / Mean	0.8389	0.9153	0.9442	0.8315	0.8741	0.7969
2	Max	0.9211	0.9842	1.0000	0.9408	0.9483	0.9119
2	Mean	0.9368	0.9737	0.9895	0.9237	0.9567	0.9084

Note. Custom group-level metrics on 190 cross-language queries. Candidate groups are restricted to the query-specific target languages; group relevance is defined by the maximum qrel grade within a problem-language group, with grade >= 2 treated as relevant. These values are not directly comparable with official document-level CoREB benchmark metrics.

Table 4. Cluster-bootstrap differences

Aggregation	Metric	Difference (k=2 − k=1)	95% cluster-bootstrap CI
Max	Hit@1	+0.0821	[0.0511, 0.1169]
Max	Hit@5	+0.0689	[0.0409, 0.1021]
Max	Recall@10	+0.1093	[0.0748, 0.1464]
Max	MRR	+0.0742	[0.0487, 0.1036]
Max	nDCG@10	+0.1149	[0.0820, 0.1505]
Mean	Hit@1	+0.0979	[0.0662, 0.1330]
Mean	Hit@5	+0.0584	[0.0294, 0.0919]
Mean	Recall@10	+0.0922	[0.0597, 0.1287]
Mean	MRR	+0.0826	[0.0563, 0.1129]
Mean	nDCG@10	+0.1114	[0.0815, 0.1448]

Note. Differences are computed from the unrounded per-query results; small discrepancies from subtracting the rounded values in Table 3 are therefore possible. The 95% intervals use 5,000 percentile-bootstrap replicates (seed = 20261031), with the 2.5th and 97.5th percentiles as interval bounds. The 61 source-problem clusters are defined by meta_source_problem_id; all queries associated with a sampled problem are retained together. These are custom group-level metrics, not official document-level CoREB benchmark scores.

For both aggregation strategies, all five 95% source-problem cluster-bootstrap intervals for the k=2 minus k=1 contrast lie above zero (Table 4). Under Max, the estimated differences range from +0.0689 for Hit@5 to +0.1149 for nDCG@10. Under Mean, they range from +0.0584 for Hit@5 to +0.1114 for nDCG@10. For Hit@1, the difference is +0.0821 (95% CI [0.0511, 0.1169]) for Max and +0.0979 (95% CI [0.0662, 0.1330]) for Mean. These estimates support an improvement from one to two references under this group-level protocol, conditional on the evaluated set of source problems.

The point-estimate comparison between Max and Mean is mixed. At k=2, Mean has higher Hit@1 and MRR, while Max has higher Hit@5 and Recall@10. The bootstrap analysis in Table 4 does not test whether Max and Mean differ significantly from each other; it tests k=2 versus k=1 separately within each operator. Almost all CoREB groups contain two implementations, so this dataset does not meaningfully test reference sets larger than two. The group-level labels also collapse the benchmark's original document-level judgments; these custom metrics must not be compared directly with official CoREB leaderboard metrics.

4.3. Document-level CoREB baseline

For context, the original document-level TF–IDF baseline over the 190 cross-language queries produced Hit@1 = 0.1842, Hit@5 = 0.6105, Hit@10 = 0.6684, Recall@10 = 0.5013, MRR = 0.3574, and nDCG@10 = 0.3444. These values describe ranking individual documents across the corpus. They are not directly comparable to the higher scores in Table 3, which ranks groups after restricting candidates to query-specific target languages and fixing relevance at the group level.

4.4. Reference filtering and candidate-space effects

In the unrestricted filtering comparison, restricting the corpus to implementations marked as having passed tests improves retrieval metrics while substantially reducing the candidate space. With one random reference, Hit@1 increases from 0.8284 to 0.9047 and MRR from 0.8681 to 0.9320. The average candidate group count falls from 395.13 to 151.45. A source-problem cluster bootstrap over 61 clusters gives a Hit@1 difference of +0.0763 (95% CI [0.0479, 0.1083]), an MRR difference of +0.0639 (95% CI [0.0409, 0.0899]), and an nDCG@10 difference of +0.0837 (95% CI [0.0566, 0.1147]).

With all references, the unrestricted comparison remains mixed. For Max, the passed-only minus all-reference difference is +0.0211 for Hit@1 (95% CI [0.0000, 0.0514]), +0.0148 for MRR (95% CI [0.0030, 0.0312]), and +0.0213 for nDCG@10 (95% CI [0.0120, 0.0327]). For Mean, the Hit@1 difference is +0.0158 (95% CI [−0.0057, 0.0462]) and the MRR difference is +0.0096 (95% CI [−0.0070, 0.0298]), while Recall@10 and nDCG@10 have positive intervals of [0.0087, 0.0299] and [0.0026, 0.0294]. These comparisons still change the candidate pool and therefore do not isolate reference quality.

Table 5. Matched-candidate control

Aggregation	Reference set	Hit@1	Hit@5	Recall@10	MRR	nDCG@10
Max	All	0.9421	0.9947	0.9548	0.9627	0.9320
Max	Passed only	0.9421	0.9947	0.9557	0.9631	0.9331
Mean	All	0.9421	0.9842	0.9421	0.9611	0.9231
Mean	Passed only	0.9526	0.9842	0.9425	0.9662	0.9236

Note. Both conditions rank the same candidate groups; only reference composition differs.

The matched-candidate results change little. Figure 3 displays the point-estimate differences between passed-only and all-reference conditions when the same candidate groups are ranked. In the source-problem cluster bootstrap, the Max difference for nDCG@10 is +0.0011 (95% CI [−0.0004, 0.0029]), and the Recall@10 difference is +0.0009 (95% CI [−0.0013, 0.0040]). For Mean, the Hit@1 difference is +0.0105 (95% CI [−0.0108, 0.0385]), the MRR difference is +0.0051 (95% CI [−0.0093, 0.0232]), and the nDCG@10 difference is +0.0005 (95% CI [−0.0109, 0.0135]). The bootstrap used 5,000 replicates and comparison-specific seeds 20261033 for Max and 20261034 for Mean. The intervals do not provide clear evidence of a consistent ranking benefit from passed-only references when candidate groups are held fixed.

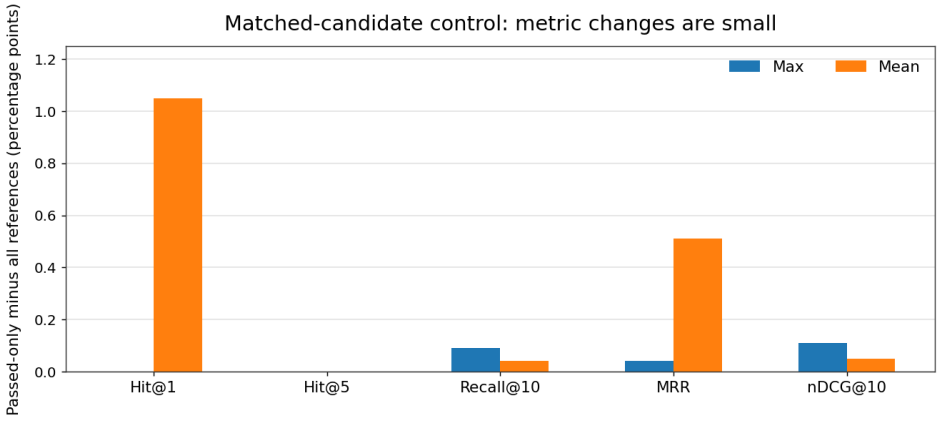


Figure 3. Matched-candidate point-estimate changes (passed-only minus all references). Differences are shown in percentage points; 95% cluster-bootstrap intervals are discussed in the text and included in the reproducibility artifacts.

This conclusion is specific to this CoREB subset and the group-level protocol. All positively judged documents in the analyzed subset were marked as passed, so filtering retained every relevant group. That property should not be assumed for other datasets. The meta_passed field indicates test status; it is not a general measure of code quality or relevance.

5. Discussion

Across both datasets, the reference-count results were positive, but the scope of the evidence differs. On AtCoder, the comparison extends from one to all available references while keeping the candidate cohort fixed. CoREB supports only the one-versus-two-reference contrast because almost all groups contain two implementations. Its cluster-bootstrap intervals are above zero for all five tested metrics under both aggregation strategies, but this result remains conditional on the evaluated subset and its group-level relevance protocol.

Mean outperformed Max on AtCoder, but CoREB results were mixed: at k=2, Mean scored higher on Hit@1 and MRR, while Max scored higher on Hit@5 and Recall@10. The bootstrap intervals test adding a reference within each operator, not Max versus Mean. Mean may favor candidates similar to several references, while Max can retain a strong match to one; this study did not test that explanation.

The filtering experiment illustrates why candidate construction matters. In the unrestricted comparison, filtering looks beneficial, but it also removes many candidate groups. Once the candidate set is held fixed, the metric changes are small. A retrieval system can therefore appear to improve because the ranking task became easier, not because the remaining reference implementations provide much stronger evidence.

The scores from AtCoder and CoREB describe different retrieval tasks. On AtCoder, a result is relevant when it belongs to the same programming task as the query. In the CoREB reference-set experiment, document judgments are combined into problem-language groups, and candidate languages are restricted for each query. The scores therefore cannot be used to claim that the system performs better on one dataset than on the other. The release includes an explicit environment specification with pinned primary packages and rerun instructions; independent reproduction in a clean environment remains a useful external check.

6. Limitations

Several limitations constrain how broadly these results can be interpreted.

Lexical representation

TF–IDF captures token overlap, not program semantics. It does not directly model abstract syntax trees, runtime behavior, or learned code representations. A stronger representation could change the relative performance of max and mean aggregation.

Task identity as relevance on AtCoder

An AtCoder candidate is relevant when it belongs to the same task as the query. This proxy is useful for task-level retrieval but does not prove that the candidate is the closest semantic equivalent or the best implementation for a developer.

Group-level judgments on CoREB

The CoREB group-level experiment assigns each group the maximum relevance grade among its judged documents and treats grades of at least 2 as relevant. This operation loses distinctions between positive documents and hard negatives within a group. The reported group-level metrics, including binary nDCG@10, are custom metrics and are not equivalent to official document-level CoREB scores.

Cluster-bootstrap scope

CoREB reference-count intervals resample 61 source-problem clusters and keep the queries belonging to each sampled problem together. This accounts for within-problem dependence, but the inference is conditional on a limited set of source problems and the custom group-level protocol. The intervals assess k=2 versus k=1 within each aggregation strategy; they do not provide a cluster-adjusted comparison of Max directly against Mean.

Limited reference-set size and filtering labels

Almost all CoREB groups contain two implementations, restricting the analysis to one versus two references. In addition, the filtering result depends on the behavior of meta_passed and the positive judgments in this particular subset. Other collections may contain relevant solutions that fail available tests or may lack reliable test-status labels.

Generalisability

The study covers two datasets and one lexical retrieval baseline. Results may differ for other languages, repositories, query distributions, embeddings, or candidate-generation strategies. Future work should compare this baseline with BM25 and evaluate additional controls that hold both the candidate set and reference count constant. The experiments support empirical claims about the tested settings, not a claim that a particular configuration is optimal for all code search systems.

Reproducibility artifacts

The CoREB data are pinned to revision e3f40f2609b4c52cc6d750594a87b29b370c3e06 of hq-bench/coreb (split release_v2603). The CoREB one-reference conditions use seeds 0–9. Its reference-count bootstrap uses 5,000 percentile replicates (seed 20261031; 2.5th and 97.5th percentile bounds) over 61 source-problem clusters. The filtering bootstrap uses 5,000 percentile replicates over the same 61 source-problem clusters, with base seed 20261030 and comparison seeds 20261030–20261034. In the comparison order, these correspond to unrestricted one-passed minus one-any, unrestricted all-passed minus all-any for Max, the same comparison for Mean, matched-candidate passed versus all references for Max, and the matched-candidate comparison for Mean. Random-reference seeds are averaged within each query before bootstrap differences are calculated; each cluster draw retains all paired queries for each sampled source problem. For AtCoder, the fixed-cohort reference-selection seeds are 0–4; reference-count contrasts use 10,000 task-level bootstrap replicates (seed 20261028), and task-weighted Max-versus-Mean contrasts use 5,000 task-level replicates (seed 20261028). The AtCoder archive matched the expected MD5 checksum 11e95fae502b9fffcd9771aa1d9cb01a, and ZIP integrity testing found no damaged member. The release now includes the analysis notebook, an explicit rerun environment specification with pinned primary packages, a run guide, the bootstrap output tables, and file checksums. The historical Colab runtime package versions were not recorded in the original notebook metadata; the environment specification supplied here is a reproducibility target for reruns, not a claim that it exactly recreates the original runtime.

7. Conclusion

The results suggest that the number of reference implementations matters for lexical cross-language code retrieval. On the fixed AtCoder cohort, adding references improved retrieval performance, and mean aggregation outperformed maximum aggregation at every tested multi-reference size. CoREB also showed improvements when moving from one reference to two under both aggregation strategies, although its group-level evaluation does not establish whether the pattern would continue with larger reference sets.

The filtering experiment was more mixed. Restricting the corpus to implementations marked as having passed tests improved the unrestricted comparison, but the differences became small when both conditions ranked the same candidate groups. In this setting, much of the apparent improvement was associated with reducing the candidate space rather than with a clear advantage from passed-only references.

Taken together, these findings point to reference-set construction as a design choice worth testing rather than a universally optimal rule. The next step is to check whether these patterns hold under an independent reproduction, an alternative lexical baseline such as BM25, and additional datasets.

References

[1] G. Mathew and K. T. Stolee, “Cross-Language Code Search using Static and Dynamic Analyses,” in Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’21), 2021. doi: 10.1145/3468264.3468538. https://doi.org/10.1145/3468264.3468538

[2] R. Yoshimura, K. Sakamoto, H. Washizaki, and Y. Fukazawa, “WOJR: A Recommendation System for Providing Similar Problems to Programming Assignments,” Applied System Innovation, vol. 5, no. 3, article 53, 2022. doi: 10.3390/asi5030053. https://doi.org/10.3390/asi5030053

[3] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, “XLCoST: A Benchmark Dataset for Cross-lingual Code Intelligence,” arXiv:2206.08474, 2022; presented at the ICLR 2023 Workshop on Deep Learning for Code. https://arxiv.org/abs/2206.08474. Workshop version: https://openreview.net/forum?id=CuP9bgvsKA

[4] S. Xue, Z. Liao, J. Qin, Z. Zhang, Y. Mu, F. Zhou, and H. Yu, “Beyond Retrieval: A Multitask Benchmark and Model for Code Search,” arXiv:2605.04615, 2026. https://arxiv.org/abs/2605.04615. Dataset and benchmark documentation: https://huggingface.co/datasets/hq-bench/coreb

[5] “Cross-language code search using static and dynamic analyses” [Dataset], Zenodo, record 4002869, version published February 23, 2021. DOI: 10.5281/zenodo.4002869. https://doi.org/10.5281/zenodo.4002869. File: AtCoder.mongo.zip; expected MD5: 11e95fae502b9fffcd9771aa1d9cb01a.